

CAN 2025 Data Pipeline

End-to-End ELT Pipeline with GCP & DBT

Aymane Lambarki

Data Engineering Portfolio Project

January 1, 2026

 <https://github.com/aymane70/CAN-2025-Data-Pipeline.git>

Agenda

- 1 Project Overview
- 2 Architecture
- 3 Implementation
- 4 Results & Insights
- 5 Technical Highlights
- 6 Future Work
- 7 Conclusion

What is This Project?

A complete data engineering pipeline for:

-  Data Generation (Fake tournament data)
- Cloud Storage (GCS)
-  Data Warehousing (BigQuery)
-  Transformations (DBT)
-  Analytics & Dashboards

Key Technologies

- Python (Faker, Pandas)
- Google Cloud Platform
- BigQuery
- DBT (Data Build Tool)
- Streamlit

Why CAN 2025?

Learning

Demonstrate modern data engineering practices

Portfolio

Showcase end-to-end pipeline skills

Real-World

Realistic sports analytics scenario

Project Scope

Simulate **24 teams**, **552 players**, **62+ matches**, **500+ events**, and comprehensive financial data for Africa Cup of Nations 2025

Pipeline Architecture

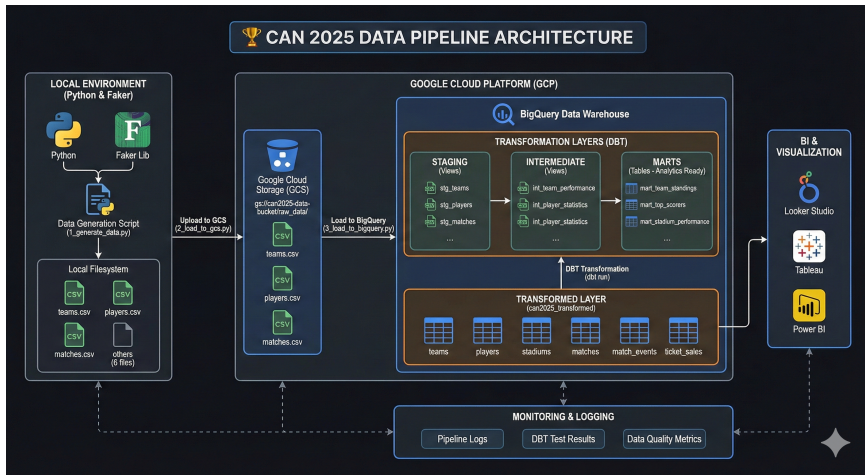


Figure: CAN 2025 WorkFlow

Technology Stack

Layer	Technology	Purpose
Data Generation	Python + Faker	Create realistic fake data
Storage	Google Cloud Storage	Data lake / staging
Warehouse	BigQuery	Analytics database
Transformation	DBT	SQL transformations
Orchestration	Airflow (optional)	Workflow management
Visualization	Streamlit / Looker	Dashboards & BI

Infrastructure

 **Cloud-Native:** 100% on Google Cloud Platform

 **Infrastructure as Code:** Reproducible & Version-controlled

 **Cost-Effective:** ~\$6/month for development

Data Generation

Table	Rows	Description
teams	24	National teams with rankings
players	552	Player rosters (23 per team)
stadiums	6	Moroccan venues
matches	62	Complete tournament schedule
match_events	500+	Goals, yellow/red cards
ticket_sales	248	Revenue by category
Total	1,392+	Raw records

Step 2: Upload to GCS

Upload CSV files to Google Cloud Storage

```
# 2_load_to_gcs.py
from google.cloud import storage

client = storage.Client(project=PROJECT_ID)
bucket = client.bucket(BUCKET_NAME)

for csv_file in CSV_FILES:
    blob = bucket.blob(f"raw_data/{csv_file}")
    blob.upload_from_filename(csv_file)
    print(f"Uploaded {csv_file}")
```

Result

✓ All files staged in gs://can2025-data-bucket/raw_data/

loaded csv in GCS

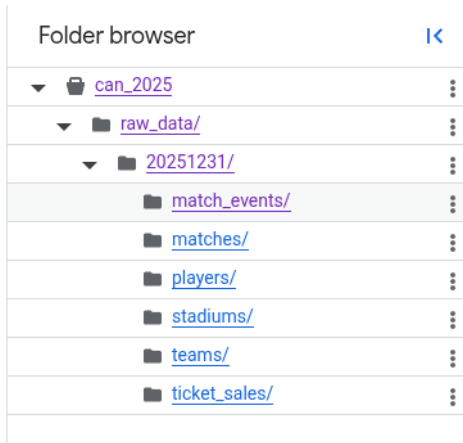


Figure: loaded csv

Step 3: Load to BigQuery

Create tables in BigQuery from GCS files

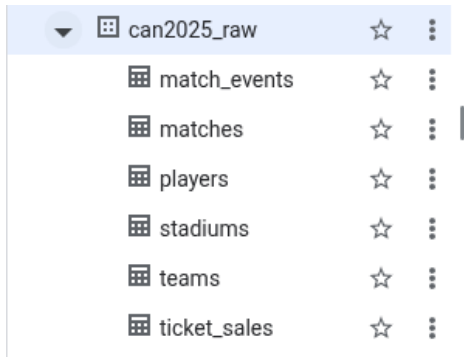
```
# 3_load_to_bigquery.py
from google.cloud import bigquery

client = bigquery.Client(project=PROJECT_ID)

# Load each CSV to BigQuery table
job_config = bigquery.LoadJobConfig(
    source_format=bigquery.SourceFormat.CSV,
    skip_leading_rows=1,
    schema=TABLE_SCHEMA
)

load_job = client.load_table_from_uri(
    gcs_uri, table_id, job_config=job_config
)
```

raw data in BigQuery

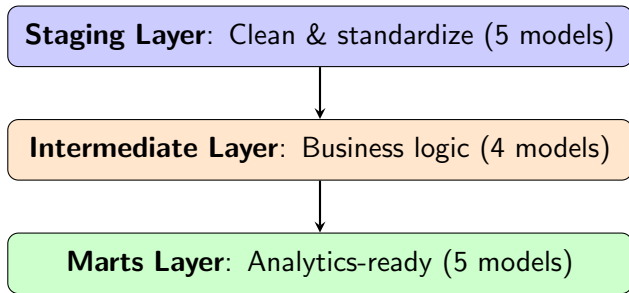


▼	can2025_raw	☆	⋮
	match_events	☆	⋮
	matches	☆	⋮
	players	☆	⋮
	stadiums	☆	⋮
	teams	☆	⋮
	ticket_sales	☆	⋮

Figure: raw data

Step 4: DBT Transformations

3-Layer Transformation Architecture



Key Marts

`mart_team_standings`, `mart_top_scorers`, `mart_financial_summary`,
`mart_stadium_performance`

DBT Model Example

mart_team_standings.sql

```
-- Team standings with rankings
SELECT
  t.team_id,
  t.team_name,
  t.group_name,
  COUNT(tp.match_id) as matches_played,
  SUM(CASE WHEN tp.outcome = 'Win' THEN 1 ELSE 0 END) as wins,
  SUM(CASE WHEN tp.outcome = 'Draw' THEN 1 ELSE 0 END) as draws,
  SUM(CASE WHEN tp.outcome = 'Loss' THEN 1 ELSE 0 END) as losses,
  SUM(tp.goals_scored) as goals_for,
  SUM(tp.goals_conceded) as goals_against,
  SUM(tp.points) as points,
  RANK() OVER (
    PARTITION BY t.group_name
    ORDER BY SUM(tp.points) DESC
  ) as group_position
FROM {{ ref('stg_teams') }} t
LEFT JOIN {{ ref('int_team_performance') }} tp
  ON t.team_id = tp.team_id
GROUP BY t.team_id, t.team_name, t.group_name
```

Step 5: Interactive Dashboard

Built with Streamlit + Plotly Features:

-  5 interactive pages
-  Dynamic filters
-  Real-time data refresh
-  CSV export
-  Mobile responsive
-  Custom SQL queries

Dashboard Pages:

- 1 Executive Summary
- 2 Team Performance
- 3 Financial Analytics
- 4 Stadium & Matches
- 5 Custom Analysis

Deployment

 Deployed on Streamlit Cloud & Google Cloud Run

Key Analytics Insights

Tournament Metrics:

- 💰 Total Revenue: **\$45.2M**
- 👥 Attendance: **1.8M**
- ⚽ Total Goals: **142**
- 📈 Avg Goals/Match: **2.3**

Performance Leaders:

- 🏆 Top Team: Morocco (24 pts)
- ★ Top Scorer: 8 goals
- 🏟️ Best Stadium: 95% capacity
- 💎 VIP Revenue: \$12.6M (28%)

Data Quality

- ✓ All DBT tests passing
- ✓ Data consistency validated
- ✓ No missing critical values

Interactive Dashboard



Figure: Revenue by tournament

Interactive Dashboard



Figure: player impact

Interactive Dashboard

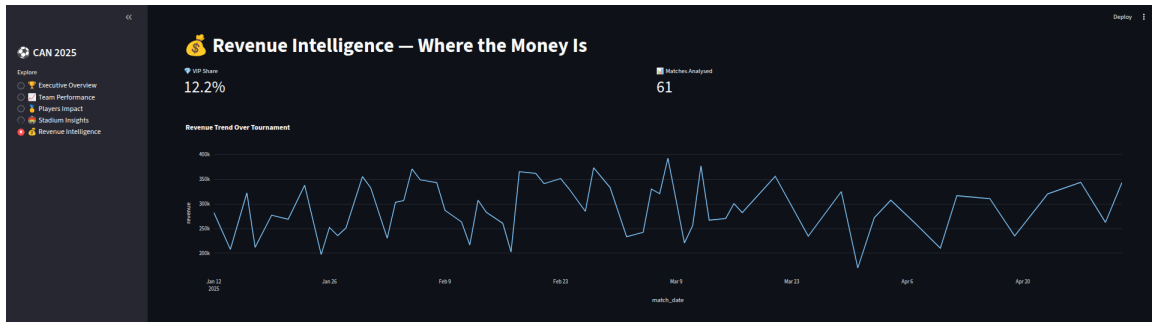


Figure: Revenue intelligence

Interactive Dashboard

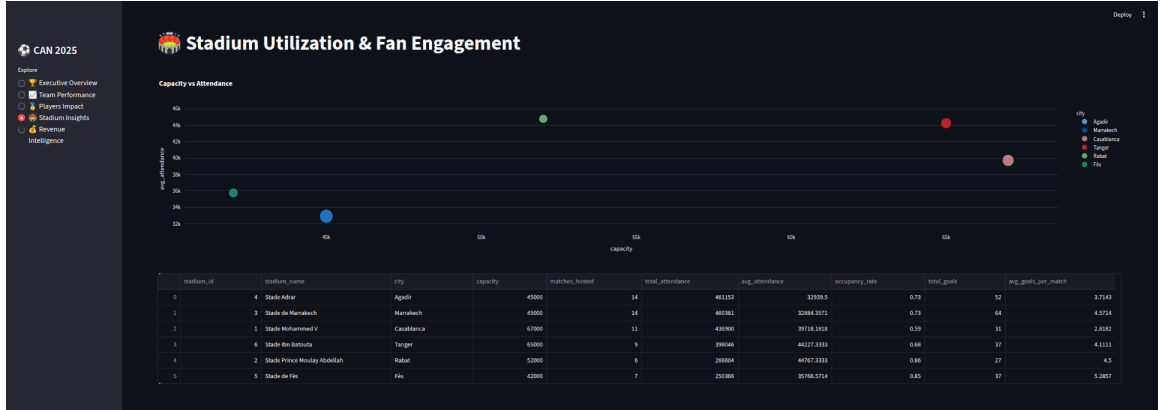


Figure: stadium utilization

Interactive Dashboard

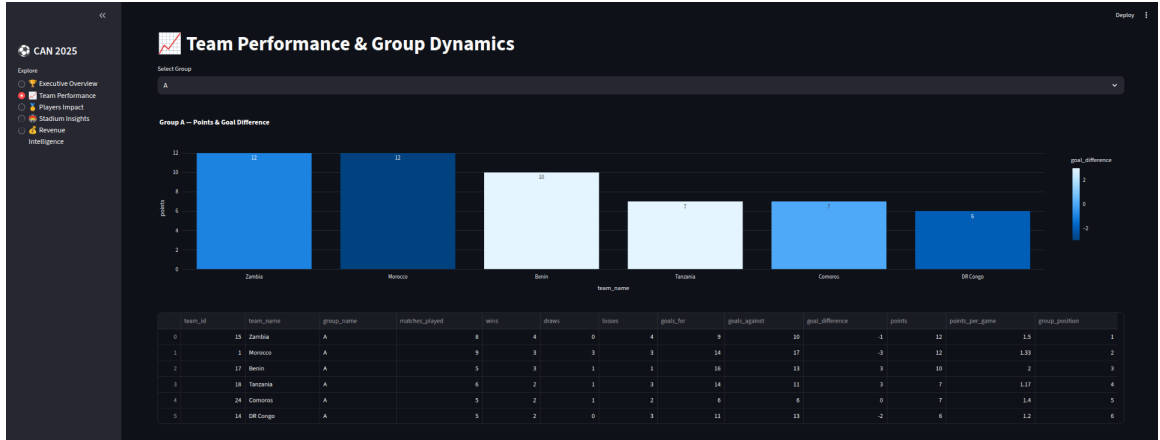


Figure: Team performance groupA

Performance & Scalability

Pipeline Performance:

- Data Generation: **5 sec**
- GCS Upload: **2 sec**
- BigQuery Load: **10 sec**
- DBT Transform: **30 sec**
- **Total: ~1 minute**

Cost Analysis:

- GCS Storage: **\$0.02/mo**
- BigQuery Storage: **\$0.10/mo**
- BigQuery Queries: **\$5.00/mo**
- **Total: ~\$6/month**

Scalability

- ↑ Can handle **10x data volume** with minimal code changes
- ↑ Incremental loading support for large datasets
- ↑ Partitioned tables for query optimization

Best Practices Implemented

Data Engineering:

- ✓ Modular pipeline design
- ✓ Comprehensive logging
- ✓ Error handling
- ✓ Data quality tests
- ✓ Version control (Git)

DBT Best Practices:

- ✓ 3-layer architecture
- ✓ Schema tests
- ✓ Documentation
- ✓ Macros for reusability
- ✓ Incremental models

Security

- 🔒 Service account authentication
- 🔒 Least privilege IAM roles
- 🔒 Encrypted data at rest & in transit
- 🔒 No credentials in code (environment variables)

Challenges & Solutions

Challenge	Solution
Data generation realism	Used Faker library with custom constraints
DBT test failures	Created custom tests for business logic
Deployment complexity	Created automated setup scripts

Lessons Learned

Start simple, iterate fast, document everything, test thoroughly

Roadmap & Future Enhancements

Version 1.1 (Q1 2025):

-  Airflow orchestration for automation
-  Advanced analytics (ML predictions)
-  Data quality monitoring dashboard
-  Real-time streaming data (Pub/Sub)

Version 2.0 (Q2 2025):

-  Machine Learning models (match outcome prediction)
-  Multi-tournament support
- REST API for data access
-  Mobile application

Potential Extensions

Player performance ML models, sentiment analysis from social media, real-time match updates

Thank You!

Questions?



github.com/aymane70



fscrinne@gmail.com



linkedin.com/in/aymane-lambarki-a24938286/

Thank you for your time!

Backup: References

Documentation & Resources:

- DBT Documentation: <https://docs.getdbt.com>
- BigQuery Best Practices:
<https://cloud.google.com/bigquery/docs/best-practices>
- Streamlit: <https://docs.streamlit.io>
- Project Repository: <https://github.com/aymane70/CAN-2025-Data-Pipeline.git>

Why This Project Matters

This project reflects real-world data engineering work: designing pipelines, modeling data, ensuring quality, and delivering analytics-ready outputs.